



Programming of Safety Critical Systems

Programming and Development of Embedded Systems

Safety Critical Systems

❖ Safety Critical Systems

- Systems whose failure or malfunction may result in one (or more) of the following
 - Loss of life or serious injury to people
 - Damage to properties/equipments
 - Damage to the environment

❖ Examples of safety critical systems

- Aircraft control systems
- Robotic surgery machines
- Railway signal control systems
- Braking system in vehicles

❖ Safety-critical systems are increasingly computer-based



Safety Critical Systems

- ❖ **Hazard:** A condition which can cause harm or damage
- ❖ **Risk:** Probability that a system causes harm or damage in the case of hazardous event
- ❖ **Functional Safety** is absence of **unreasonable** risk of harming people or damaging the environment due to hazards caused by malfunctioning behaviour of systems
- ❖ **Dependability** is a general term
 - **Reliability:** Continuity of correct service.
 - **Availability:** Readiness for correct service.
 - **Safety:** Absence of harming people and damaging the environment.
- ❖ **Safety vs. Security** (or cyber security)
 - **Safety:** Absence of internal failures which can cause harm or damage
 - **Security:** Protection against external threats which can cause harm or damage
 - You can have Security without Safety. But, in most cases, not Safety without Security.

Safety Critical Systems

- ❖ Risk can never be completely eliminated. Probabilities can be reduced.
- ❖ Development of Safety Critical Systems
 - Creating systems that always operate safely by minimizing, controlling and reducing hazards
 - And in the case of failure they are still capable of entering in a controlled safe operation mode
 - Is performed according to the **state of the art** in all aspects of safety.
 - Reducing risks as far as "reasonably practicable" - good enough.
 - The burden of proof is with the manufacturer.
 - Documented proof must be produced.
 - International standards are required.
- ❖ There are different standards in different industries.
 - IEC 61508 for industry in general
 - IEC 62304 for medical industry
 - ISO 26262 for road vehicle industry

Safety Critical Systems

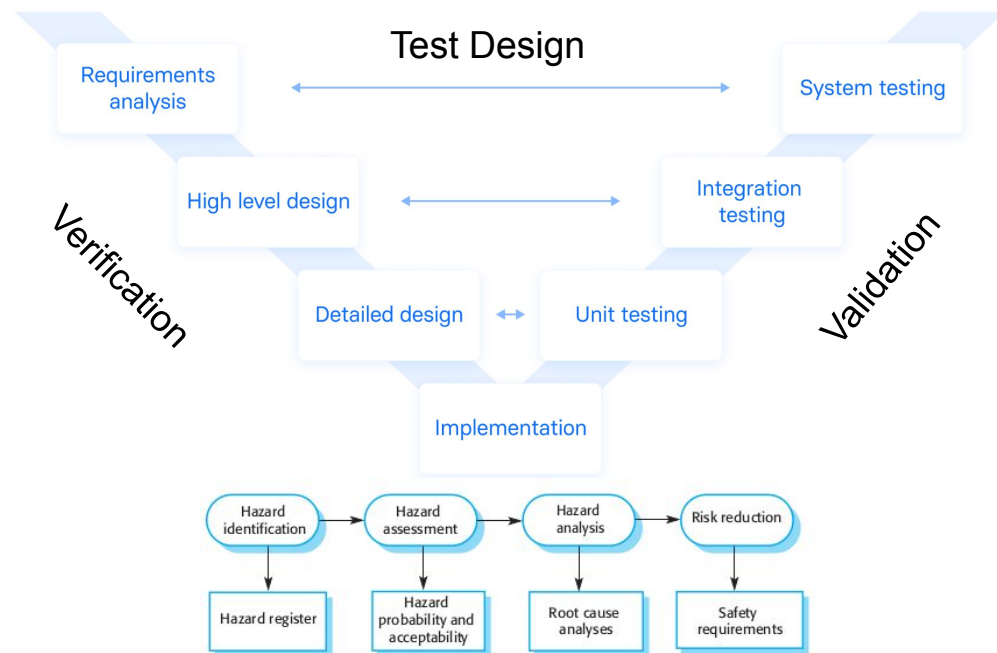
❖ Types of Safety Critical System in Term of Reliability

- **Fail-operational** systems: Remaining part of the system can complete the operation
 - E.g. a remote keyless system
- **Fail-safe** systems: They switch to a safe mode and usually inform an operator
 - E.g. Windows, Insulin pumps and etc.
- **Fail-secure** systems: They become secure usually by locking up to minimize harm
 - E.g. Electronic doors, lock during power failures
- **Fail-passive** systems: They become passive and hand over control to an operator;
 - E.g. An aircraft autopilot
- **Fault-tolerant** systems: They continue to operate in the case of failure
 - Usually by detecting the at risk components and replace them before they result in any risk
 - E.g. cooling system in a nuclear reactor



Development of Safety Critical Software Systems

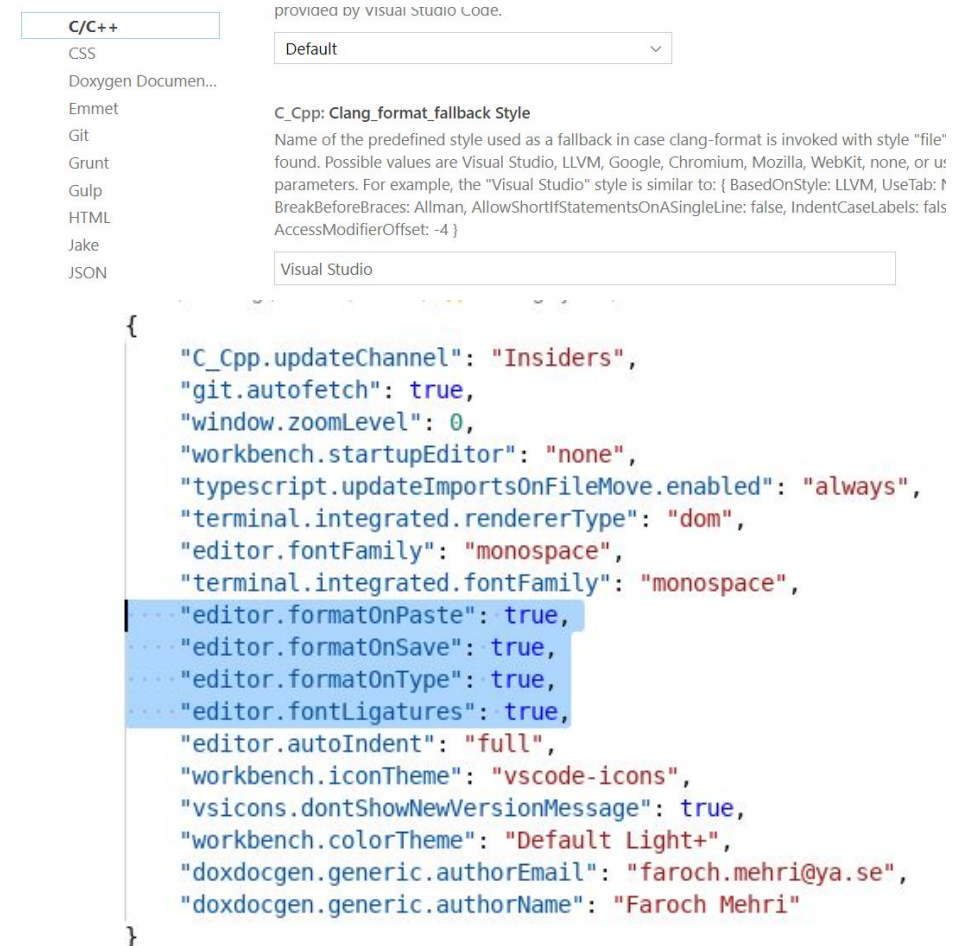
- ❖ At **system, hardware** and **software** levels, the classic V-Model framework is used
- ❖ Standards like ISO 26262, AUTOSAR, MISRA C, and etc. are used.
 - MISRA C defines a subset of C language to be used in programming of safety critical systems
 - ISO 26262 is the international functional safety standard for road vehicles.
- ❖ Development at the software level
 - Safety requirements Specification.
 - Architectural design.
 - Unit design and implementation.
 - Unit testing.
 - Integration testing
 - System testing
 - Verification and validation of safety requirements in each phase.



Safety Critical Software Systems - Guidelines

❖ Modelling and coding guidelines

- Enforcement of low complexity
- Use of language subsets
- Enforcement of strong typing
- Use of defensive implementation techniques
- Use of well-trusted design principles
- Use of unambiguous graphical representation
- Use of naming conventions
- Use of style guides
 - Like Visual Studio Style Guides for C/C++
 - Activate it in Visual Studio Code



Safety Critical Software Systems - Guidelines

- ❖ Principles for software architectural design
 - Hierarchical structure of software components
 - Restricted size and complexity of software components
 - Restricted size of interfaces
 - Strong cohesion within each software component
 - Loose coupling between software components
 - Appropriate scheduling properties (timing property of the components)
 - Appropriate management of shared resources
 - Appropriate isolation of the software components
 - Restricted use of interrupts
 - Use of design notation, like UML, flow charts and structure charts

Safety Critical Software Systems - Guidelines

- ❖ Design principles for software unit design and implementation
 - Use of semi-formal notation; like UML, timing diagram and etc.
 - One entry and one exit point in subprograms and functions
 - No dynamic objects or variables, or else online test during their creation
 - Initialization of variables
 - No multiple use of variable names
 - Avoid global variables or else justify their usage
 - Restricted use of pointers
 - No implicit type conversions
 - No hidden data flow or control flow
 - No unconditional jumps
 - No recursions

Safety Critical Software Systems - Guidelines

❖ Methods for software verification

- Walk-through
- Pair-programming
- Semi-formal verification
- Control flow and data flow analysis
- Static code analysis; like [Coverity](#), [Helix QAC](#), [IAR](#), [CodeSonar](#), [Parasoft C/C++test](#) and etc.
- Requirement-based test
- Interface test
- Fault injection test
- Resource usage evaluation
- Analysis of boundary values

Safety Critical Software Systems - Guidelines

❖ Structural coverage metrics

- Statement coverage
- Branch coverage
- Modified condition/decision coverage (MC/DC)
 - Each entry and exit point are invoked
 - Each decision takes every possible outcome
 - Each condition in a decision takes every possible outcome
 - Each condition in a decision is shown to independently affect the outcome of the decision.
 - Independence of a condition is shown by proving that only one condition changes at a time.
- Function coverage
- Call coverage

Safety Critical Software Systems - Guidelines

❖ Doxygen

- The de facto standard tool to generate documentation from C, C++ and etc. source files
- It generates the documents based on the comments and descriptions in source files
- Install **Doxygen Documentation Generator extension** in Visual Studio Code

test Struct Reference

My test struct. More...

Data Fields

int **foo**
int **bar**
My bar field.

Detailed Description

My test struct.

Field Documentation

int test::foo
My foo field. foo is important

```
int amount = readNumber("CPUs", 1, 10);  
  
Cpu MyCpu(type, 0, a  
MySystem.AddModule(4  
response = readChar  
switch (response) {  
    case 'Q':  
        return 2; //  
    case 'R':  
        type = Disk;  
        break;  
    case 'S':  
        default :  
            Parameter:  
            item Type of item  
            Parameter:  
            min Least amount of item customer can order  
            Parameter:  
            max Maximum amount of item customer can order  
            Returns:
```

commands

documentation

```
348  
349  
350  * @brief disable GPIO wake-up function.  
351  * @param gpio_num: GPIO number  
352  * @return ESP_OK: success  
353  *         ESP_ERR_INVALID_ARG: parameter error  
354  
355  esp_err_t gpio_wakeup_disable(gpio_num_t gpio_num);  
356
```

function prototype

Function Documentation

daxpy()

```
auto daxpy ( size_t      n,  
             double const * xs,  
             double *    ys,  
             double      alpha,  
             double      beta  
             )  
            -> void
```

C++ implementation of Fortran BLAS daxpy
Computes the equation $ys[i] \leftarrow xs[i] * \alpha + \beta * ys[i]$.

Note
Function with C-linkage.

Parameters

[in]	n	Array size. Size of xs and ys
[in]	xs	Input array xs
[in, out]	ys	Output array ys
[in]	alpha	Linear coefficient
	beta	Void

Side Effects and Sequence Points

- ❖ The evaluation order of operands is undefined for most of the operators in C
 - Therefore some unexpected results may occur when using certain operators
 - The results may be different in different compilers and different machines
 - The undefined behaviours can make programs unportable
 - These unexpected results are caused by **side effects**
- ❖ Any operation that affects storage of an operand has a **side effect**
- ❖ Any operation has a **persistent side effect** if it does anything beyond its life
- ❖ Conflicting **side effects** are a kind of undefined behavior.
- ❖ C guarantees that all side effects of an expression will be concluded by the next **sequence point** in a program.

```
int x = 20;
int f1(void) { x += 10; return x; }
int f2(void) { x -= 5; return x; }
int main(void) {
    // 30 + 25 or 15 + 25
    int p = f1() + f2();
    printf("p = %d", p);
    return 0;
}
```

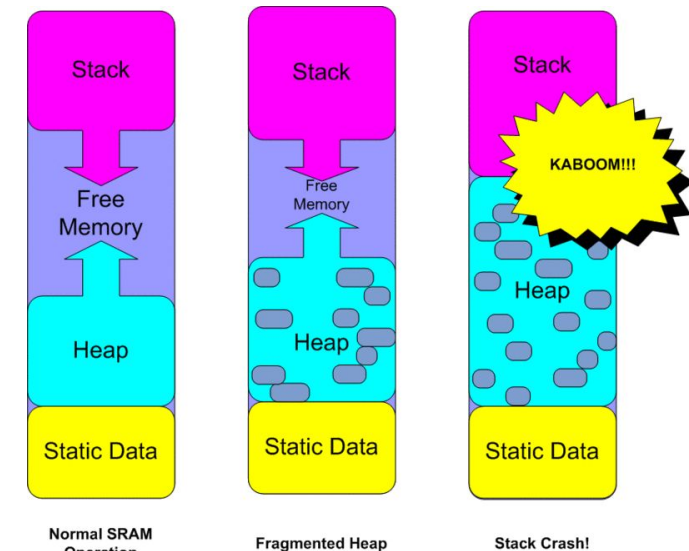
```
int x = 3; printf("%d, %d, %d, %d\n", x++, x, ++x, x);
```

Side Effects and Sequence Points

- ❖ **Sequence points** are checkpoints in a program at which the compiler ensures that operations in an expression are concluded.
- ❖ The most known sequence point is the **semicolon** marking the end of a statement.
- ❖ Some other sequence points are as follows:
 - `expr1, expr2` (the comma operator)
 - `expr1 && expr2` (the logical AND operator)
 - `expr1 || expr2` (the logical OR operator)
 - `expr1 ? expr2 : expr3` (the conditional operator)
- ❖ Using the execution order of side effects, when none is guaranteed, is risky
- ❖ Best practice: no more than one side effect on an object in an expression.

Dynamic Memory Allocation & Deallocation

- ❖ The main purpose of dynamic memory allocation & deallocation
 - Multiple processes can dynamically share all the available RAM memory
 - E.g. a multi-process system with a large and variable or unknown amount of memory
- ❖ In addition to memory leak, there are some problems with dynamic memory allocation/deallocation
 - It might fail
 - Memory fragmentation
 - It is slow; memory needs to be scanned to find free space
 - Memory management is required
 - Its execution time is non-deterministic and varying
 - In the case of time-critical systems it is unacceptable



Dynamic Memory Allocation & Deallocation

- ❖ Dynamic memory allocation is not safe, secure and deterministic.
- ❖ Generally dynamic memory allocation/deallocation is not used in safety critical systems:
 - It can be harmful and it is hard to handle failures
 - Safety critical systems are deterministic (no dynamic behavior)
 - You always know the required amount of memory for the worst case
- ❖ Alternatives for dynamic memory allocation/deallocation
 - Static memory allocation
 - Static memory preallocation and a safe memory management implementation
 - E.g. A memory pool or a stack-based memory management
 - Memory allocation at startup is generally safe; but not freeing
 - Creating single-instance modules which can manage a known number of sets of data by determining the number of required instances and statically allocating memory for them

MISRA C Guidelines

❖ Guideline categories in MISRA C

- Mandatory guidelines - C code which is claimed to conform to MISRA C
 - Shall comply with every mandatory guideline
 - Deviation from mandatory guidelines is not permitted
- Required guidelines - C code which is claimed to conform to MISRA C
 - Shall comply with every required guideline
 - In the case of deviation, a formal deviation is required
 - Such deviations are properly recorded and authorized
 - Individual programmers can be prevented from deviating guidelines by a formal authorization
- Advisory guidelines
 - These are recommendations. It does not mean that these items can be ignored, but rather that they should be followed as far as they are reasonably practical.
- *The Standard Library is not required to comply with MISRA C.*

MISRA C Guidelines

❖ The essential type model

- Operands shall not be of an inappropriate essential type

Operator	Operand	Essential type category of arithmetic operand					
		Boolean	character	enum	signed	unsigned	floating
[]	integer	×	×				×
+ (unary)		×	×	×			
- (unary)		×	×	×		×	
+ -	either	×		×			
* /	either	×	×	×			
%	either	×	×	×			×
< > <= >=	either	×					
== !=							

MISRA C Guidelines

❖ The essential type model ...

Operator	Operand	Essential type category of arithmetic operand					
		Boolean	character	enum	signed	unsigned	floating
! &&	any		×	×	×	×	×
<< >>	left	×	×	×	×		×
<< >>	right	×	×	×	×		×
~ & ^	any	×	×	×	×		×
?:	1st		×	×	×	×	×
?:	2nd and 3rd						

- Expressions of essentially character type shall not be used inappropriately in addition and subtraction
 - Exceptions: convert between digits in the range '0' to '9' and the corresponding ordinal value
 - Convert a character from lowercase to uppercase and vice versa

MISRA C Guidelines

- ❖ If a function returns error information, then the error information shall be tested
- ❖ The content of a header file shall not be included more than once
- ❖ Validity of values passed to library functions shall be checked
- ❖ Dynamic memory allocation shall not be used
- ❖ A program shall contain no violations of the standard C
- ❖ There shall be no occurrence of undefined or critical unspecified behaviour
- ❖ The character sequences `/*` and `//` shall not be used within a comment
 - Exception: The sequence `//` is permitted within a `//` comment
- ❖ Functions and objects shall not be defined with external linkage if they are referenced in only one translation unit (file)

MISRA C Guidelines

- ❖ Identifiers that define objects or functions with external linkage shall be unique

```
/* file1.c */
int32_t count; /* "count" has external linkage */

void foo(void) /* "foo" has external linkage */
{
    int16_t index; /* "index" has no linkage */
}

/* file2.c */
static void foo(void) /* Non-compliant - "foo" is not unique (it is already defined with external linkage in file1.c) */
{
    int16_t count; /* Non-compliant - "count" has no linkage but clashes with an identifier with external linkage */
    int32_t index; /* Compliant - "index" has no linkage */
}
```

- ❖ An external object or function shall be declared once in one and only one source file
- ❖ An identifier with external linkage shall have exactly one external definition

```
/* file1.c */
int16_t i = 10;

/* file2.c */
int16_t i = 20; /* Non-compliant - two definitions of i */

/* featureX.h */
extern int16_t a; /* Declare a */

/* file.c */
#include "featureX.h"
int16_t a = 0; /* Define a */
```

MISRA C Guidelines

- ❖ The **static** storage class specifier shall be used in all declarations of objects and functions that have internal linkage
- ❖ An **inline function** shall be declared with the static storage class

```
extern int32_t array1[10]; /* Compliant */  
extern int32_t array1[];  /* Non-compliant */
```
- ❖ When an array with external linkage is declared, its size shall be explicitly specified
- ❖ Within an enumerator list, the value of an implicitly-specified enumeration constant shall be unique

```
/* Non-compliant - yellow replicates implicit green */  
enum colour { red = 3, blue, green, yellow = 5 };  
  
/* Compliant */  
enum colour { red = 3, blue, green = 5, yellow = 5 };
```
- ❖ Functions shall not call themselves, either directly or indirectly
- ❖ The value returned by a function having non-void return type shall be used
- ❖ A macro shall not be defined with the same name as a keyword
- ❖ All **#else**, **#elif** and **#endif** preprocessor directives shall reside in the same file as the **#if**, **#ifdef** or **#ifndef** directives to which they are related

MISRA C Guidelines

- ❖ A reserved identifier or macro name shall not be declared
- ❖ The standard memory allocation/deallocation functions of `<stdlib.h>` shall not be used
- ❖ The standard header file `<setjmp.h>` shall not be used
- ❖ The standard header file `<signal.h>` shall not be used
- ❖ The standard library input/output functions shall not be used
- ❖ The `atof`, `atoi`, `atol` and `atoll` functions of the standard library `<stdlib.h>` shall not be used
- ❖ The standard library functions `abort`, `exit`, `getenv` and `system` of `<stdlib.h>` shall not be used
- ❖ The standard library functions `bsearch` and `qsort` of `<stdlib.h>` shall not be used
- ❖ The standard library time and date functions shall not be used
- ❖ All resources obtained dynamically by the standard library functions shall be explicitly released
- ❖ The same file shall not be open for read and write access at the same time on different streams

MISRA C Guidelines

❖ The essential type model

- The value of an expression should not be cast to an inappropriate essential type
- Casting from **void** to any other type is not permitted as it results in **undefined behaviour**.

Essential type category	from					
to	Boolean	character	enum	signed integer	unsigned integer	floating
Boolean		×	×	×	×	×
character	×					×
enum	×	×	×	×	×	×
signed integer	×					
unsigned integer	×					
floating	×	×				

MISRA C Guidelines

- ❖ If a pointer to a structure or union is never dereferenced within a translation unit, then the implementation of the object should be hidden
- ❖ Functions which are designed to provide operations on a resource should be called in an appropriate sequence. e.g. opening a file; reading from a file; closing a file;
- ❖ A project should not contain unused type declarations
- ❖ A project should not contain unused macro declarations
- ❖ An object should be defined at block scope if its identifier only appears in a single function
- ❖ A pointer should not be converted into an integer and vice versa.
 - If casting between integers and pointers is used, care should be taken. E.g. addressing memory mapped registers or other hardware specific features

Programming and Development of Safety Critical Systems

❖ Some useful links

- [Safety Critical System](#)
- [Safety critical sensors in cars](#)
- [Automotive functional safety](#)
- [Safe and Reliable Computer Control Systems Concepts and Methods](#)
- [Functional Safety with ISO 26262 - Principles and Practice](#)
- [GENIVI Alliance](#)
- [Autonomous Vehicle Standards](#)
- [CERT C Programming Language Secure Coding Standard](#)
- [A Quick Guide to ISO 26262](#)
- [Doxygen](#)